JournaBuddy: System Architecture & Dynamic Visual Report

Comprehensive "For Dummies" Explanatory Guide with TikZ Diagrams

Abdullah Al Mamun
BSc, MSc - Software Engineering
TU Wien (Vienna, Austria) & Daffodil International University
Email: mamun.swe.de@gmail.com — GitHub: @abbysweb
ORCID: 0009-0006-7473-0024

July 29, 2026

Contents

1 Introduction: What is JournaBuddy?

JournaBuddy is an artificial intelligence platform designed to help scientific authors prepare, optimize, and evaluate their research papers before submitting them to academic journals.

Instead of guessing whether a manuscript will be rejected due to poor formatting, weak citations, or out-of-scope journal selection, JournaBuddy runs automated checks that evaluate the paper across three core layers:

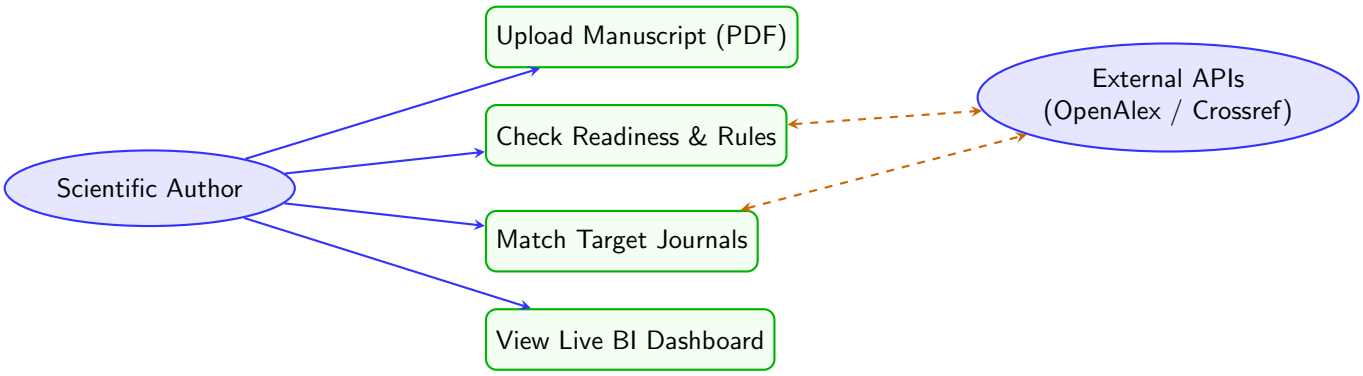
1. **Symbolic / Rule-Based Layer:** Checks exact facts (structure, acronym definitions, citation formatting).
2. **Statistical NLP Layer:** Calculates mathematical text scores (readability, passive voice, jargon density).
3. **Semantic / AI Layer:** Uses local LLM agents and vector search to evaluate methodology and journal scope fit.

—

2 System Diagrams & Visual Architecture

2.1 1. Use Case Diagram

The Use Case Diagram shows all primary interactions between scientific authors, external academic services, and the JournaBuddy system.



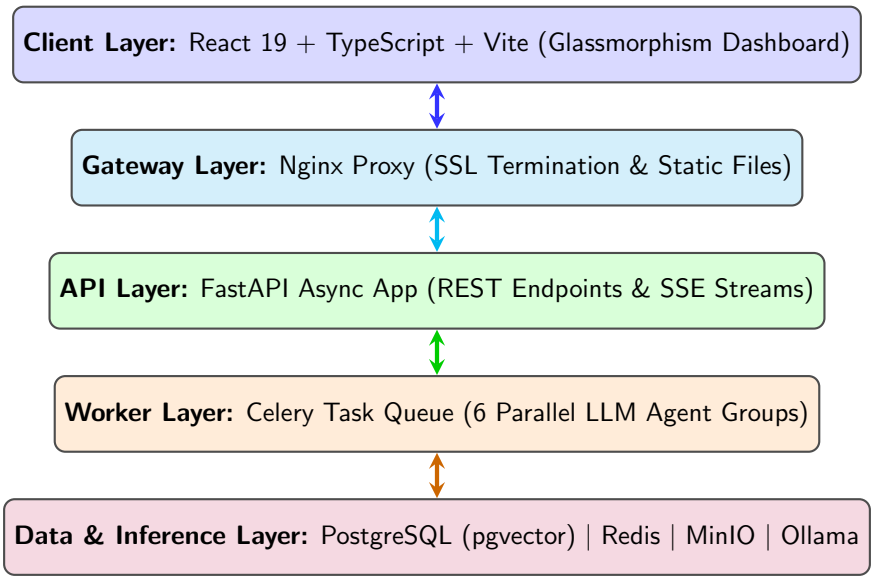
Explanation "For Dummies": Use Case Diagram

Think of this diagram as a restaurant menu showing what users can order. The **Scientific Author** (on the left) can perform four major actions: uploading a manuscript, checking readiness, finding target journals, and viewing the live dashboard. The **External APIs** (on the right) work in the background like food suppliers, providing extra data (like citation counts and journal trust metrics) whenever requested.

—

2.2 2. System Architecture Diagram

The System Architecture Diagram displays the multi-tier containerized stack powering JournaBuddy.



Explanation "For Dummies": System Architecture

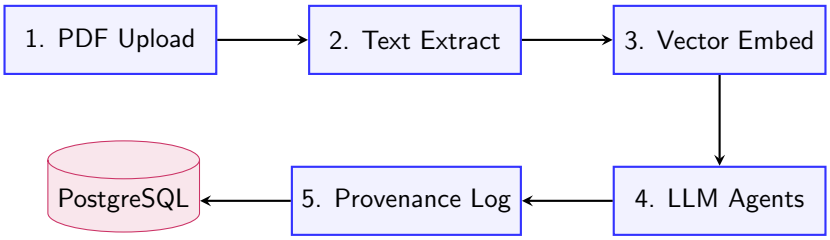
Imagine JournaBuddy as a 5-story building:

1. **Top Floor (Client):** The visual web interface where you click buttons and see graphs.
2. **4th Floor (Gateway):** The security guard (Nginx) that filters bad requests and directs traffic.
3. **3rd Floor (API):** The main manager (FastAPI) who receives your uploaded paper and creates processing jobs.
4. **2nd Floor (Workers):** A team of assistant workers (Celery) who do heavy calculations in parallel.
5. **Basement (Data & Brain):** The vault holding your database (PostgreSQL), memory cache (Redis), file storage (MinIO), and local AI brain (Ollama).

—

2.3 3. Data Flow Diagram (DFD)

The Data Flow Diagram tracks how an uploaded paper moves through processing stages and transforms into score reports.



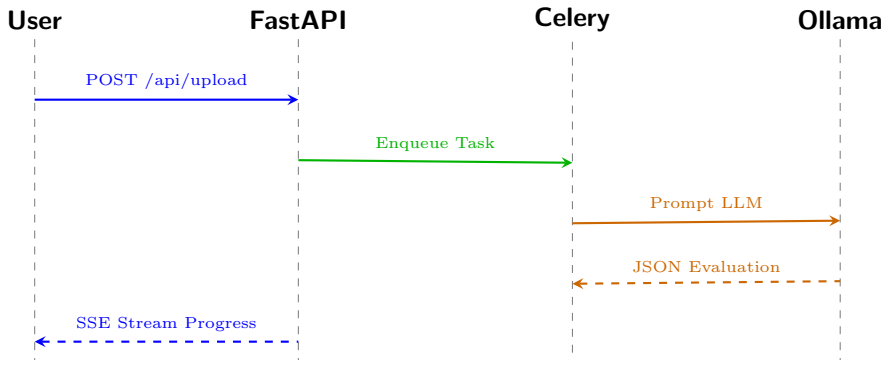
Explanation "For Dummies": Data Flow Diagram

This flow shows a paper's journey step by step: First, your PDF is received (**Step 1**). Next, the text is extracted from the pages (**Step 2**). That text is converted into numbers called vectors (**Step 3**). The AI agents read those numbers and evaluate your paper (**Step 4**). Finally, every score formula is recorded (**Step 5**) and safely stored in the database.

—

2.4 4. Sequence Diagram

The Sequence Diagram displays the exact order of network messages between components during a live upload.



Explanation "For Dummies": Sequence Diagram

This is a timeline showing who talks to whom in real-time: 1. You click "Upload", sending your file to FastAPI. 2. FastAPI hands the job to Celery so your browser doesn't freeze waiting. 3. Celery sends prompts to Ollama to review the text. 4. Ollama responds with scores and feedback. 5. FastAPI streams the progress right back to your screen live!

—

3 Continuous Implementation Roadmap

- **Phase 1 (Infrastructure):** Docker Compose, FastAPI, MinIO, Vite React setup. (Status: *Complete & Verified*)
- **Phase 2 (AI Pipeline):** Celery workers, Ollama Llama 3.1 integration, vector embeddings.
- **Phase 3 (External Enrichment):** OpenAlex, Crossref, DOAJ integrations.
- **Phase 4 (Real-time UI):** Glassmorphism dashboard, SSE streaming, interactive charts.
- **Phase 5 (Hardening):** Redis caching, Prometheus/Grafana monitoring, security audits.